



1803

# INFORMÁTICA II

*Bioingeniería*

# UNIDAD #2

Computación numérica con  
Numpy, Matplotlib, Scipy,  
Pandas



UNIVERSIDAD  
DE ANTIOQUIA  
Facultad de Ingeniería



**SciPy**

SciPy es una colección de algoritmos matemáticos y funciones de conveniencia construidas sobre la extensión **NumPy** de Python. Agrega un poder significativo a la sesión interactiva de Python al proporcionar al usuario comandos y clases de alto nivel para **manipular y visualizar** datos.

SciPy es una biblioteca de cálculo científico que utiliza **NumPy**.

Proporciona más funciones de utilidad para **optimización, estadísticas y procesamiento de señales**.

Al igual que NumPy, SciPy es de código abierto por lo que podemos usarlo libremente.



# Instalación e Importación

```
conda install scipy
```

```
py - m pip install scipy
```

```
import scipy
```



# PAQUETES

| Sr. | Sub-Package         | Description                                                 |
|-----|---------------------|-------------------------------------------------------------|
| 1.  | scipy.cluster       | Cluster algorithms are used to vector quantization/ Kmeans. |
| 2.  | scipy.constants     | It represents physical and mathematical constants.          |
| 3.  | scipy.fftpack       | It is used for <b>Fourier transform</b> .                   |
| 4.  | scipy.integrate     | Integration routines                                        |
| 5.  | scipy.interpolation | Interpolation                                               |
| 6.  | scipy.linalg        | It is used for linear algebra routine.                      |
| 7.  | scipy.io            | It is used for data input and output.                       |
| 8.  | scipy.ndimage       | It is used for the n-dimension image.                       |
| 9.  | scipy.odr           | Orthogonal distance regression.                             |
| 10. | scipy.optimize      | It is used for optimization.                                |

# PAQUETES

| Sr. | Sub-Package   | Description                              |
|-----|---------------|------------------------------------------|
| 11. | scipy.signal  | It is used in signal processing.         |
| 12. | scipy.sparse  | Sparse matrices and associated routines. |
| 13. | scipy.spatial | Spatial data structures and algorithms.  |
| 14. | scipy.special | Special Function.                        |
| 15. | scipy.stats   | Statistics.                              |
| 16. | scipy.weaves  | It is a tool for writing.                |



# scipy.constants

| Physical Constants | Description                        |
|--------------------|------------------------------------|
| c                  | Speed of light in vaccum           |
| speed_of_light     | Speed of light in vaccum           |
| G                  | Standard acceleration of gravity   |
| G                  | Newton Constant of gravitation     |
| E                  | Elementry charge                   |
| R                  | Molar gas constant                 |
| Alpha              | Fine-structure constant            |
| N_A                | Avagadro constant                  |
| K                  | Boltzmann constant                 |
| Sigma              | Stefan-Boltzmann constant $\sigma$ |
| m_e                | Electron mass                      |
| m_p                | Proton mass                        |
| m_n                | Neutron Mass                       |
| H                  | Plank Constant                     |

**El paquete `scipy.constant` proporciona la cantidad de constantes físicas. Las constantes físicas más utilizadas son las siguientes:**

# scipy.constants



```
In [1]: from scipy import constants as C
```

```
In [2]: C.c # Velocidad de la luz en m/s
```

```
Out[2]: 299792458.0
```

```
In [4]: C.e # Carga del electrón
```

```
Out[4]: 1.602176565e-19
```

```
In [6]: C.atmosphere
```

```
Out[7]: 101325.0
```

```
In [7]: C.mmHg
```

```
Out[7]: 133.32236842105263
```

```
In [8]: C.Julian_year
```

```
Out[8]: 31557600.0
```

```
In [9]: C.Avogadro
```

```
Out[9]: 6.02214129e+23
```

```
In [10]: C.parsec
```

```
Out[10]: 3.0856775813057292e+16
```

```
In [11]: C.Stefan_Boltzmann
```

```
Out[11]: 5.670373e-08
```

```
In [13]: C.convert_temperature(np.array([0, 100.0]), 'Celsius', 'Kelvin') # Conversor temps
```

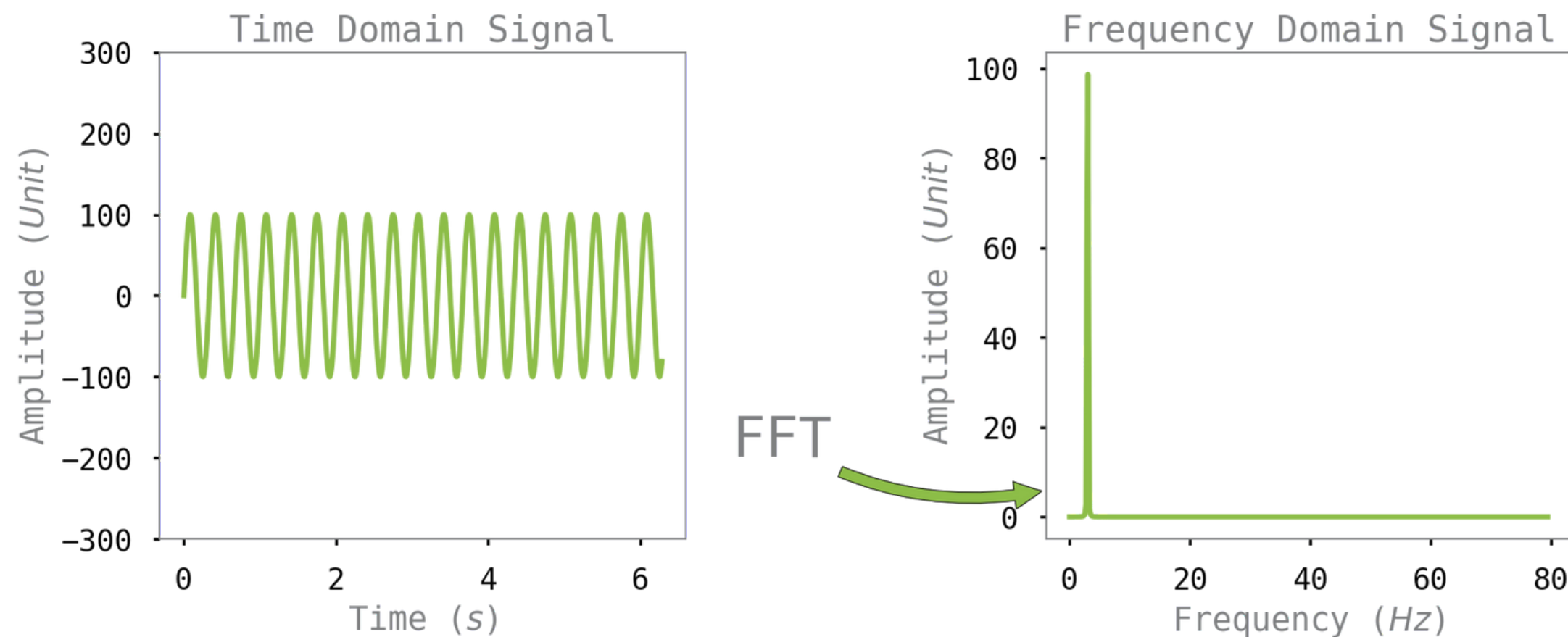
```
Out[13]: array([ 273.15,  373.15])
```

```
In [14]: C.day # Dia en segundos
```

```
Out[14]: 86400.0
```

# SciPy FFTpack

La FFT significa **Transformación Rápida de Fourier**. La transformada de Fourier convierte la señal en el dominio del tiempo al dominio de la frecuencia. Divide una forma de onda (una función o señal) en otra sustitución caracterizada por seno y coseno.



SciPy proporciona el módulo `fftpack`, que se utiliza para calcular la transformación de Fourier.

# SciPy FFTpack

```
# importing the fft and inverse fft functions from fftpackage
from scipy.fftpack import fft
#Importing numpy
import numpy as np
#create an array with random n numbers
x = np.random.rand(20)
#Applying the fft function
y = fft(x)
print(y)
```

# Transformada rápida de Fourier

## Transformada de Fourier discreta unidimensional

**La transformada discreta de Fourier (DFT) es la transformada discreta más importante y se utiliza para realizar el análisis de Fourier en muchas aplicaciones prácticas.**



# Interpolación

La interpolación se define como **encontrar un valor entre dos puntos en una línea o curva**. La primera parte de la palabra es "inter" que significa "entrar", lo que nos indica que miremos dentro de los datos. En otras palabras, " la estimación del valor intermedio entre los puntos de datos precisos se denomina interpolación ". La interpolación es muy útil en estadística, ciencia y negocios o cuando es necesario predecir el valor que existe dentro de dos puntos de datos existentes.



# Interpolación

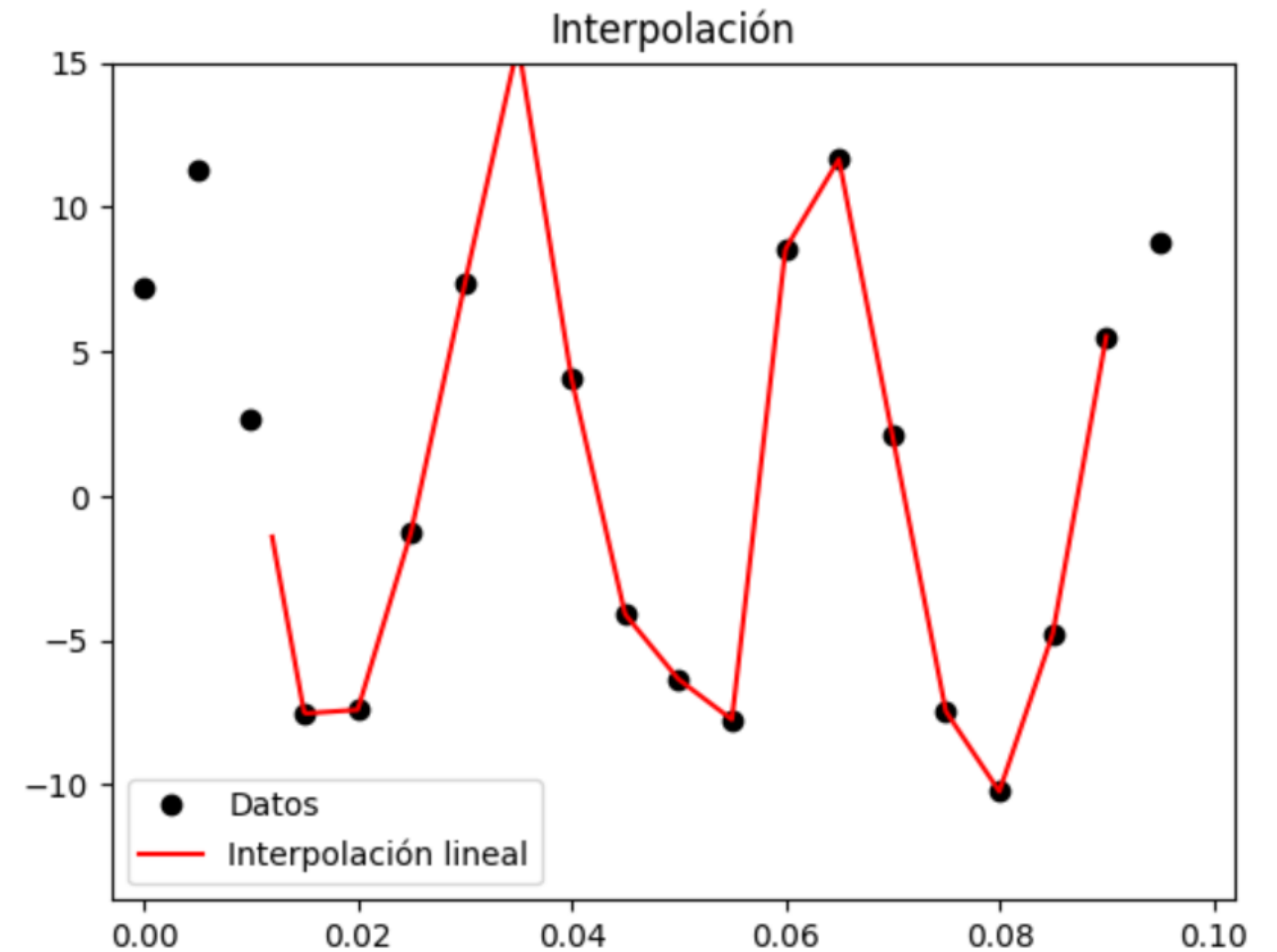
```
from scipy.interpolate import interp1d
import matplotlib.pyplot as plt

datos_x = np.arange(0, 0.1, 0.005)
A, k, theta = 10.0, 33.3, np.pi/5.0
y_real = A*np.sin(2*np.pi*k*datos_x + theta)
datos_y = y_real + 2*np.random.randn(len(datos_x))

interpolador_lineal = interp1d(datos_x, datos_y)

x_inter = np.linspace(0.012, 0.09, 500)
y_inter_l = interpolador_lineal(x_inter)

plt.plot(datos_x, datos_y, 'ok', label="Datos")
plt.plot(x_inter, y_inter_l, 'r', label="Interpolación lineal")
plt.title("Interpolación")
plt.xlim(-0.003, 0.102)
plt.ylim(-14, 15)
plt.legend()
```



# Inputs y outputs de SciPy

El paquete SciPy.io (entrada y salida) admite una amplia gama de funciones para trabajar con diferentes formatos de archivos.

```
import scipy.io as sio
#Guardar un archivo .mat
sio.savemat('python.mat', {'A1': [0, 1, 2], 'A2': 0})

#Cargar un archivo .mat
mat_file_content = sio.loadmat('python.mat')
print(sio.whosmat('python.mat'))
print(mat_file_content['A1'])
mat_file_content['A1'].mean()
```

# GRACIAS